Q1

(1) No the CFLs is not closed under complement.

Firstly we define two context free Languages.

$L_1 = \{a^n b^n c^m \mid n \geq 0, m \geq 0\}$

$L_2 = \{a^m b^n c^n \mid m \geq 0, n \geq 0\}$

they are both context-free cause we could write the grammar for it

L_1: $S \to AB$, $A \to aAb \mid \varepsilon$ $B \to cB \mid \varepsilon$

L_2: $S \to CA$, $C \to aC \mid \varepsilon$ $A \to bAc \mid \varepsilon$.

Secondly $L_1 \cap L_2$ the string w is ① $w \in L_1$ whas the form $a^n b^n c^m$ for m, n

② $w \in L_2$ whas the form $a^m b^n c^n$ for m, n

so we could have $L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 0\} = L_0$

we call this language L_0.

Third, we assue L_0 is context-free, let p be the pumping length.

$S = a^p b^p c^p \in L_0$; Because $|S| = 3p \geq p$ the pumping Lemma applies:

write $S = uvxyz$ satisfying $|vxy| \leq p$ and $|vy| \geq 1$.

Case A:

vxy is inside one block (only a's or b's or c's)

WLOG suppose v and y consist only of a's (the other two are similar)

we pump down $i=0$ we get string uxz. the decreases number of a's so we have $a = p - t$; $b = p$; $c = p$; for some $t \geq 1$ because $|vy| \geq 1$. these three counts are not equal. $uxz \notin L_0$ contradicts.

Case B:

vxy: cross the a-b boundary.

Let r_a be the number of a's in vy, and r_b be the number of b's in vy. we have $r_a + r_b \geq 1$ pumping $i=0$ removes those letters.

$a = p - r_a$ $b = p - r_b$ $c = p$.

· we would need all three counts equal:

$p - r_a = p - r_b = p$.

for $p - r_a = p$ and $p - r_b = p$ r_a and r_b must be 0 but · $r_a + r_b \geq 1$.

contradicts.

Case C:

vxy cross the b-c boundary.

This is similar to case B, (pump i=0 and the counts of b and c but · can not still equal to unchanged number of a's).

Case D:

|vxy| cross the a,b,c boundary, this is impossible because of · pumping-lemma constraint $|vxy| \leq p$. so it contradict.

All possible locations leads to contradictions with the pumping lemma's requirements.

So that $\{a^n b^n c^n \mid n \geq 0\}$ is not CFL

we found two CFLs L_1 and L_2 whose interection L_0 which is not CFL.

So CFLs is not closed under intersections.

So we could use de Morgon to show that CFLs are not closed under complement.

assume that CFLs are closed under complement, we know that CFLs are closed under union so for any CFLs A and B

$A \cap B = \overline{\bar{A} \cup \bar{B}}$; because it is not closed under intersection, ~~and~~ but CFLs closed under union · which contradict our assumption

Therefore · CFLs cannot be closed under complement.

CS 扫描全能王
3亿人都在用的扫描App

(2) Yes the class Turing recognizable Language is closed under concatenation

As we know that Language L is in Turing recognizable iff $\overline{L}$ is recursively enumerable

Proof:

Let $L_1, L_2 \subseteq \Sigma^*$ be arbitray Languages with $L_1, L_2 \in$ co-RE (Turing co-recognizable lannages)

$$L_1 \cdot L_2 = \{uv \mid u \in L_1, v \in L_2\}.$$

Because $L_1, L_2 \in$ co-RE, there exist Turing Machine M_1 and M_2 such that

$$dom(M_1) = \overline{L_1} \qquad dom(M_2) = \overline{L_2}$$

we will construct a Turing machine M that recognizes $\overline{L_1 L_2}$ since recognizability of the complement implies in co-RE proves closure.

Firstly

on input $w \in \Sigma^*$, M do.

1. enumerate all spilts of w into two pieces: $w = u_1 u_2$
2. For each ~~splitt~~ split $w = u_1 u_2$, Stimulate M_1 on u_1 and M_2 on u_2 in an interleaved fashion: at step t perform one step of each active simulation. cycling through all spilts and both machines.
3. If for a give splits $u_1 u_2$ we observe that either M_1 accepts u_1 or M_2 accepts ~~u_2~~ u_2, mark that splits as refused.
4. If every split of w becomes refused, then M accepts w.
5. Otherwise M continues the dovetailed simulation forever.

$\Rightarrow$ for $w \notin L_1 \cdot L_2$: By definition: for every split $w = u_1 u_2$ at least one of $u_1 \notin L_1$ or $u_2 \notin L_2$ holds; thus for every splits either $u_1 \in \overline{L_1}$ or $u_2 \in \overline{L_2}$. Therefore for each split, M_1 on u_1 or M_2 on u_2 is guaranteed to accept at finite time. M will eventually observe a acceptance. for every split and will accept w. Hence $w \in dom(M)$

$\Rightarrow$ If $w \in L_1 \cdot L_2$. There exist split $w = u_1 u_2$ with $u_1 \in L_1$ and $u_2 \in L_2$

For that split neither $u_1 \in \overline{L_1}$ nor $u_2 \in \overline{L_2}$ holds, so neither M_1 on u_1 nor M_2 on u_2 will ever accept. Hence $w \notin dom(M)$.

From the two point we have

$$dom(M) = \overline{L_1 \cdot L_2}$$

So $\overline{L_1 \cdot L_2}$ is Turing recongnizable., so it is closed under concatenation

Question 2.

Yes A_{CFG} is decidable.

$\Rightarrow$ Step 1. $a \in L(G)$

Convert G into an equivalent CNF : G_{CNF}.

If G generate ε, store this separetly.

To check whether $a \in L(G)$, it suffices enumerate all sequance of exactly $2n-1$ productions from G_{CNF}.

for each sequence, simulate the derivation.

If sequence derives the string a then $a \in L(G)$.

If none, $a \notin L(G)$.

If step 1 rejects, the overall rejects.

$\Rightarrow$ Step 2: $L(G) \setminus \{a\} = \emptyset \Leftrightarrow L(G) \cap (\Sigma^* \setminus \{a\}) = \emptyset$.

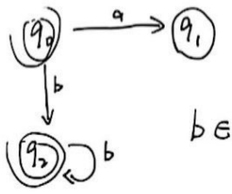
we could construct $\Sigma^* \setminus \{a\}$.

The Language $\Sigma^* \setminus \{a\}$, we can have a DFA.

DFA, $D = (Q, \Sigma, \delta, q_0, F)$. $Q = \{q_0, q_1, q_2\}$.

initial state q_0 ; $F = \{q_0, q_2\}$ $\delta(q_0, a) = q_1$ for any $b \in \Sigma$ and $b \neq a$.

$\delta(q_0, b) = q_2$; for any $b = \Sigma$ $\delta(q_1, b) = q_2$, for any $b \in \Sigma$ $\delta(q_2, b) = q_2$.



$b \in \Sigma$ and $b \neq a$.

cause CFL is closed under intersection.. $L(G') = L(G) \cap (\Sigma^* \setminus \{a\})$.

Emptiness of $L(G')$.

CFG emptiness is decidable

R = On input $\langle G' \rangle$, where G' is a CFG.

1. Mark all terminal symbols in G'
2. Repeat until no new variables get marked.
3. Mark any variable A where G' has a rule $A \to u_1 u_2 \cdots u_k$ and each symbol $u_1 \cdots u_k$ has been marked
4. If the start variable is not marked accept ; otherwise reject.

In other word $L(G') = \emptyset$ generates no string except a.

$L(G') \neq \emptyset$ then G generate some $w \neq a$, reject.

~~If the machine~~ rejects: either Step 1. fond $a \notin L(G)$ or step 2. $w \neq a$ in $L(G)$

in both $L(G) \neq \{a\}$

$\Rightarrow$ the algorithm always halts cause.

1. CNF conversion
2. Enumeration of derivation with $2n-1$
3. DFA construction.
4. CFG intersection
5. CFG emptiness.

these are all finite.

If $a = \varepsilon$. it is also proved on step 2. the emptiness of CFLs.

Therefore A_{CFG} is decidable.

Question 3.

No - T is undecidable

If we know a undecidable problem can be reduced to this language then it is undecidable.

We assume that $\overline{HALT} = \{\langle M, w\rangle \mid M$ does not halt on input $w\}$.

It is known that $\overline{HALT}$ is undecidable because of definition.

then to construct a mapping f for any $\langle M, w\rangle$:

$\langle M, w\rangle \in \overline{HALT}$ if and only if $f(\langle M, w\rangle) = \langle M'\rangle \in T$.

Input any string x

1. simulate M on input w
2. If during simulation M halts on w, then M' accepts x; otherwise M' simulates forever and never accepts x.

Correctness of the reduction.

Case1: M does not halt on w

for any input x, M' in stimulating M forever and never enters an accept state

Hence M' accepts no string: $L(M') = \emptyset$

since $L(M')$ is empty, the "all accepted strings satisfy $|w| \leq 50$ and are accepted within 50 steps" holds vacuously.

Therefore $\langle M'\rangle \in T$.

Case 2: M halts on w

Then for any input x, M' simulates M on w until it halts and then accepts x.

Thus, M' accepts every string: $L(M') = \Sigma^*$

since Σ^* contains string longer than 50, there violating $|w| \leq 50$.

Hence, $\langle M'\rangle \notin T$.

So we have: $\langle M, w\rangle \in \overline{HALT} \iff \langle M'\rangle \in T$.

Because $\overline{HALT}$ is undecidable and we have reduced it to T, the Language T is also undecidable